Project 1 — Real Estate System

CMPE250 — Fall 2026

SAs: Alperen İzbudak, Baran İnci
TA: Beyza İrem Urhan
Due: 06.11.2026 , 22.00

1 Introduction

You have just started a new job as a software developer at a local real estate agency. On your first morning, your manager pulls you aside and explains the situation: the agency has been in business for over twenty years, but lately clients have been walking out the door and signing with competitors. The problem, she says, is not the properties — the agency has hundreds of listings at any given moment — it is the software.

The current system was written long ago and has not kept up. When a client sits down and describes what they are looking for — a maximum budget, a minimum floor area — the system takes an embarrassingly long time to respond, sometimes returning stale results that include houses already sold or whose prices have since changed. Sellers, too, are frustrated: updating a listing price or removing a property from the market should be instant, but the old system sometimes lags behind.

The office's workflow is straightforward. New properties arrive throughout the day; each listing carries a unique identifier, an asking price V, and a floor area S in square metres. Clients browse the catalogue through search sessions: they specify a budget and a size requirement, and the system returns the best matching options. Once a client decides to buy, the listing is removed from the market and the session is closed. Sellers may update a listing's price at any time, or pull it from the market entirely.

Your task is to replace the old system with one that handles all of these operations correctly and efficiently, no matter how many listings are active or how quickly they change.

2 General Description

Commands are read from `input.txt` one line at a time. Each command follows a strict format so that administrators can audit every action. Your program must write the corresponding output to `output.txt`. Depending on the command, a single input line may produce one or more output lines.

Properties enter the system with three attributes: a unique ID, a price (V), and a floor area (S, in m^2). Clients search for properties by specifying a maximum price and a minimum floor area. Sellers may raise or lower a listing's price, or remove it from the system entirely.

The core goal is to present clients with the most suitable available listings, prioritising those that match their criteria, while avoiding data inconsistencies — such as selling a property that has already been removed or whose price has since changed.

3 Houses

Each house in the system is described by the following attributes:

Attribute	Symbol	Description
Identifier	`ID`	A unique alphanumeric string. No two houses share the same ID.
Price	V	A positive integer representing the listing price.
Floor area	S	A positive integer representing the size of the house in m^2.

A seller registers a house by providing its ID, price, and floor area. Multiple houses may share the same (V, S) pair; they are distinguished by their IDs. When several houses have identical price and area, they are ordered by insertion time: the listing that was added earlier takes priority.

Sellers may later update a listing's price or remove it altogether. Your system must reflect these changes immediately so that no client is ever shown — or sold — a listing that no longer exists or whose price has changed.

ID uniqueness and retirement: No two houses in the system may share the same ID at any point in time. Once a house is removed (`Remove`) or sold (`Buy`), its ID is retired: a subsequent `Add` command using the same ID is treated as an error, exactly as if that ID were still active.

4 Commands

Command names are case-sensitive and written exactly as shown below. `ID` values are unique alphanumeric strings; V and S are positive integers.

4.1 House Commands

4.1.1 Add House

Format: `Add` *`ID Price Size`*

Registers a new house in the system with the given unique ID, price, and floor area. If a house with that ID already exists or has previously been removed or sold, the command fails with an error. Houses with the same (V, S) pair are accepted; they are tracked individually by ID and ordered by arrival time.

Output: `House` *`ID`* `Added Successfully`

Example:

```
Input:  Add B23K1276 1500000 50
Output: House B23K1276 Added Successfully
```

4.1.2 Remove House

Format: `Remove` *`ID`*

Removes the listing identified by `ID` from the system. If no such listing exists, the command fails with an error.

Output: `House` *`ID`* `Removed Successfully`

Example:

```
Input:  Remove B23K1276
Output: House B23K1276 Removed Successfully
```

4.1.3 Update Price

Format: `Update` *`ID NewPrice`*

Changes the price of the listing identified by `ID` to `NewPrice`. The floor area is not affected. If the listing does not exist, or if `NewPrice` is equal to the current price, the command fails with an error.

Output: `House` *`ID`* `Updated Successfully`

Example:

```
Input:  Update B23K1276 2000000
Output: House B23K1276 Updated Successfully
```

4.2 Search Commands

4.2.1 Search

Format: `Search` *`SearchID MaxPrice MinSize Count`*

Finds the best available listings for a client and saves the result under the given `SearchID`. The client specifies:

Parameter	Meaning
`SearchID`	An identifier that is unique among open search sessions.
`MaxPrice`	Upper bound on the listing price (inclusive).
`MinSize`	Lower bound on the floor area in m^2 (inclusive).
`Count`	Number of results to return.

The system selects up to `Count` listings according to the Search Priority rules described in Section 5. Every active listing belongs to exactly one of the four priority pools, so the result may include fallback listings that do not satisfy both the price and size criteria.

Listings are displayed one per line, each showing the house ID, price, floor area, and the priority pool from which it was selected, in the order they appear in the ranked result list. The priority pool is written as `P1`, `P2`, `P3`, or `P4`.

If fewer than `Count` active listings exist, the command returns all available listings in priority order and does not output an error. The command also fails with `ERROR` if `SearchID` already identifies an open search session. Otherwise, it outputs `ERROR` only when no active listing exists.

Output (one line per result):

```
<HouseID> <Price> <Size> <Priority>
<HouseID> <Price> <Size> <Priority>
...
```

Example:

```
Input:  Search A12K89H 1000000 60 5
Output:
ID1 900000  65 P1
ID2 1000000 50 P2
ID3 1200000 70 P3
ID4 1300000 65 P3
ID5 1500000 40 P4
```

4.2.2 Buy

Format: `Buy` *`SearchID Index`*

Purchases the house at position `Index` in the previously stored search result identified by `SearchID`. The listing is removed from the system and marked as sold.

Index is 0-based: the first result in the stored list has `Index` = 0, the second has `Index` = 1, and so on.

Changes to other houses in the same stored search result do not prevent the purchase. If the house at the requested `Index` is still active and has not been updated since the search was issued, the `Buy` command succeeds even if other houses in that search result have been updated, removed, or sold.

A listing is considered updated if any `Update` command has been applied to it after the search was issued — regardless of whether later updates returned the price to the value it had at search time. For example, if a house had price P at search time, was then updated to P', and then updated back to P, it is still considered updated and a `Buy` on it will fail with `ERROR`.

The command fails with `ERROR` in any of the following situations:

- `SearchID` does not refer to an open search session, or the session has already been closed.
- `Index` is out of range or does not identify a house in the stored result list.
- The house at the requested `Index` has been updated since the search was issued.
- The house at the requested `Index` has already been sold, has been removed, or is otherwise no longer an active listing.

Output: `House` *`ID`* `Sold`

Example:

```
Input:  Buy A12K89H 3
Output: House ID4 Sold
```

4.2.3 Close Search

Format: `Close` *`SearchID`*

Closes an open search session. The stored result list is discarded. If `SearchID` does not refer to an open session, the command fails with `ERROR`. Once the session is closed, its `SearchID` may be used for a new search.

Output: `Search` *`SearchID`* `Closed`

Example:

```
Input:  Close A12K89H
Output: Search A12K89H Closed
```

4.3 Query Commands

4.3.1 Count Active Listings

Format: `Count`

Returns the total number of listings currently active in the system (i.e. houses that have been added but not yet removed or sold).

Output: `Count :` *`N`*

Example:

```
Input:  Count
Output: Count : 50000
```

4.3.2 Total Active Listing Price Query

Format: `TotalPrice` *`MaxPrice MinSize`*

Computes the sum of the current prices of all active houses whose current listing price is at most `MaxPrice` and whose floor area is at least `MinSize`. Removed and sold houses are not included. If no active house satisfies both conditions, the result is zero.

Output: `TotalPrice is` *`X`*

Example:

```
Input:  TotalPrice 500000 45
Output: TotalPrice is 15000000000
```

5 Search Priority

When building the ranked result list for a `Search` command, the following rules determine which listings are selected and in what order.

Tie-breaking by Insertion Order

Whenever two listings are otherwise equal in all ranking criteria, the listing that was added to the system earlier is ranked higher. Note that an `Update` command changes a listing's price, which resets its position in the ordering as if it were a newly added listing.

Priority 1 — Listings Within Budget and Meeting Size Requirement

$$\text{Price} \leq \texttt{MaxPrice} \quad \text{and} \quad \text{Size} \geq \texttt{MinSize}$$

The primary pool consists of all listings whose price does not exceed `MaxPrice` and whose floor area meets or exceeds `MinSize`. Listings in this pool are ranked by:

1. Price ascending (cheaper listings are ranked higher),
2. Floor area descending (larger qualifying homes rank higher when prices are equal),
3. Earlier listing takes priority for any remaining ties.

Priority 2 — Fallback: Listings Below Minimum Size

$$\text{Price} \leq \texttt{MaxPrice} \quad \text{and} \quad \text{Size} < \texttt{MinSize}$$

If the primary pool does not yield `Count` results, the remaining slots are filled from a secondary pool of listings that are within budget but below the size requirement. These listings are ranked by:

1. Price ascending (cheaper listings are ranked higher),
2. Floor area descending for equal prices (listings closest to `MinSize` from below are ranked higher),
3. Earlier listing takes priority for any remaining ties.

The result list normally consists of all Priority 1 entries followed by all Priority 2 entries, truncated to `Count` items in total.

Priority 3 — Fallback: Listings Above Budget

$$\text{Price} > \texttt{MaxPrice} \quad \text{and} \quad \text{Size} \geq \texttt{MinSize}$$

If the Priority 1 and Priority 2 pools do not yield `Count` results, the remaining slots are filled from a tertiary pool of listings that are above budget but meet or exceed the size requirement. These listings are ranked by:

1. Price ascending (listings closest to `MaxPrice` from above are ranked higher),
2. Floor area descending for equal prices,
3. Earlier listing takes priority for any remaining ties.

Priority 4 — Fallback: Listings Above Budget and Below Minimum Size

$$\text{Price} > \texttt{MaxPrice} \quad \text{and} \quad \text{Size} < \texttt{MinSize}$$

If neither the price criterion nor the size criterion is satisfied, the remaining slots are filled from a fourth pool of listings that are both above budget and below the size requirement. These listings are ranked by:

1. Price ascending (the lowest-priced listings are ranked higher),
2. Floor area descending for equal prices (larger homes are ranked higher),
3. Earlier listing takes priority for any remaining ties.

The final result list consists of all Priority 1 entries followed by all Priority 2, Priority 3, and Priority 4 entries in that order, truncated to `Count` items in total. Since every active listing belongs to exactly one of these four priorities, the command outputs `ERROR` only when there are no active listings.

Summary Table

Priority	Condition	Primary Sort	Secondary Sort
1	Price $\leq$ MaxPrice $\wedge$ Size $\geq$ MinSize	Price $\uparrow$	Size $\downarrow$, then earlier listing
2	Price $\leq$ MaxPrice $\wedge$ Size $<$ MinSize	Price $\uparrow$	Size $\downarrow$, then earlier listing
3	Price $>$ MaxPrice $\wedge$ Size $\geq$ MinSize	Price $\uparrow$	Size $\downarrow$, then earlier listing
4	Price $>$ MaxPrice $\wedge$ Size $<$ MinSize	Price $\uparrow$	Size $\downarrow$, then earlier listing

5.1 Input/Output Formulation

This subsection summarizes the input and output cases for every command.

Add Input: `Add <ID> <Price> <Size>`

- Case 1 — New ID (never used before): `House <ID> Added Successfully`
- Case 2 — ID already exists or was previously used: `ERROR`

```
Input:  Add H101 900000 65
Output: House H101 Added Successfully
```

Remove Input: `Remove <ID>`

- Case 1 — Active house exists: `House <ID> Removed Successfully`
- Case 2 — House does not exist: `ERROR`

```
Input:  Remove H101
Output: House H101 Removed Successfully
```

Update Input: `Update <ID> <NewPrice>`

- Case 1 — House exists and price changes: `House <ID> Updated Successfully`
- Case 2 — House does not exist or price is unchanged: `ERROR`

```
Input:  Update H102 950000
Output: House H102 Updated Successfully
```

Search Input: `Search <SearchID> <MaxPrice> <MinSize> <Count>`

Each result is written as `<HouseID> <Price> <Size> <Priority>`. Results are taken in the order Priority 1, Priority 2, Priority 3, and Priority 4. Within each priority, price is sorted in ascending order, size in descending order, and insertion time in ascending order.

- Case 1 — `SearchID` identifies an open search session: `ERROR`
- Case 2 — The number of active houses is at least `Count`: Exactly `Count` lines are printed, starting from the best available priority.
- Case 3 — The number of active houses is less than `Count` but greater than zero: One line is printed for every active house. Returning fewer than `Count` lines is valid and does not produce `ERROR`.
- Case 4 — No active house exists: `ERROR`

Example — At least five active houses:

```
Input:  Search A12K89H 1000000 60 5
Output:
ID1 900000  65 P1
ID2 1000000 50 P2
ID3 1200000 70 P3
ID4 1300000 65 P3
ID5 1500000 40 P4
```

Example — Only three active houses:

```
Input:  Search B45M21Q 1000000 60 5
Output:
ID6 850000  70 P1
ID7 950000  50 P2
ID8 1200000 80 P3
```

Example — No active house:

```
Input:  Search C78N34R 500000 100 5
Output: ERROR
```

Buy Input: `Buy <SearchID> <Index>`

Indices are 0-based: the first result has `Index` = 0.

- Case 1 — Requested house is unchanged and active: `House <ID> Sold`. The purchase succeeds even if other houses in the same search result have been updated, removed, or sold.

- Case 2 — Search session is closed or does not exist: `ERROR`
- Case 3 — Index is out of range or has no corresponding house in the stored result: `ERROR`
- Case 4 — House at the requested index was updated after the search: `ERROR`. This applies even if subsequent updates returned the price to its value at search time.
- Case 5 — House at the requested index was sold, removed, or is no longer active: `ERROR`

```
Input:  Buy A12K89H 3
Output: House ID4 Sold
```

Close Input: `Close <SearchID>`

- Case 1 — Search session is open: `Search <SearchID> Closed`. The identifier may then be reused for a new search.
- Case 2 — Search session does not exist or is closed: `ERROR`

```
Input:  Close A12K89H
Output: Search A12K89H Closed
```

Count Input: `Count`

The command always prints the number of active houses, including zero.

```
Input:  Count
Output: Count : 3
```

TotalPrice Input: `TotalPrice <MaxPrice> <MinSize>`

The command prints the sum of the current listing prices of all active houses satisfying both Price $\leq$ `MaxPrice` and Size $\geq$ `MinSize`. Removed and sold houses are excluded. If no active house qualifies, the sum is zero.

```
Input:  TotalPrice 1000000 60
Output: TotalPrice is 1850000
```

6 Grading and Constraints

The project will be evaluated for both correctness and performance using test cases of three different sizes. Your implementation is expected to run within the specified time limits; however, a correct solution may still receive partial credit even if it exceeds those limits. Section 6.2 provides the time limits for three representative input sizes. The limits for more complex test cases, including those designed to assess hidden edge cases, may vary.

6.1 Test Cases

- `demo.txt`: A small demonstration file provided for tracing the program by hand. It is not graded.
- `input1.txt` — **Small test, separate priority pools**: approximately 10 000 commands. Searches are designed so that each result is drawn from a single priority pool.
- `input2.txt` — **Medium test, mixed commands**: approximately 200 000 commands covering all command types.
- `input3.txt` — **Large test, mixed priority pools**: approximately 500 000 commands. Each search spans multiple priority pools simultaneously.

6.2 Grading

Table 1: Test files, time limits, and point values

Example Test File	Description	Time Limit (s)	Grading
`input1.txt`	Small — separate priority pools	0.5	40
`input2.txt`	Medium — mixed commands	1	35
`input3.txt`	Large — mixed priority pools	5	25

The actual grading will be conducted using hidden test cases of similar structure and scale, but with more demanding worst-case time complexity requirements. Therefore, ensure that your solution is sufficiently optimized to handle these cases efficiently. The grades presented in the table indicate the relative weight assigned to each input size category. As mentioned earlier, the provided sample inputs are not the only test cases used in the grading process. Make sure your implementation handles all edge cases correctly.

6.3 Command and Test Distribution

Table 2: Supported command syntax

Command	Syntax	Purpose
Add	`Add <ID> <Price> <Size>`	Register a house
Remove	`Remove <ID>`	Remove an active house
Update	`Update <ID> <NewPrice>`	Change a listing price
Search	`Search <SID> <Max> <Min> <Count>`	Create a ranked result
Buy	`Buy <SID> <Index>`	Purchase a stored result
Close	`Close <SID>`	Close a search session
Count	`Count`	Count active listings
TotalPrice	`TotalPrice <Max> <Min>`	Sum qualifying prices

7 Submission

- **ZIP filename:** Submit a single ZIP archive named `<student_number>.zip`; for example, `2023000000.zip`. Files that do not follow this naming convention may

receive a penalty.

- **Archive contents:** The archive must contain the `src/` folder and all Java source files required to compile the project. Keep `Main.java` as the main class. Do not include `.class` files, generated outputs, PDFs, or unrelated files in the submission archive.

  Important: The provided `input/`, `output/`, and `demo.txt` files are for local testing. Do not submit those `input/ output/` folders.

- **Compilation and execution:** From the project directory, the program must compile and run with the following commands:

```
javac src/*.java
java -cp src Main <input_file> <output_file>
```

- **Implementation notes:**
  - The main class must be named `Main.java`.
  - The first command-line argument is the input file, and the second is the output file.
  - All submitted source files must compile together without warnings or additional libraries.
- **Before submitting,** verify the ZIP filename, source-file names, and the compilation command. Missing files and incorrect names are common causes of automatic grading failures.

8 Warnings

- Any data structure other than `ArrayList` is forbidden; you may not use any other classes from `java.util`.
- The use of `java.lang.Math` is recommended for mathematical operations.
- All source files are checked automatically for similarity with other submissions and solutions from previous years. Submit your own implementation. Academic misconduct may result in a grade penalty and disciplinary action.
- Document important methods with concise English comments and use meaningful variable names. Do not over-comment or use unnecessarily long names; clear code is important for partial grading.
- Make sure output whitespace is correct. Trailing whitespace may be disregarded, but internal spaces, capitalisation, and line breaks must match the specification exactly.
- Use a type capable of storing the complete `TotalPrice` sum without integer overflow. All command parameters remain integers unless explicitly stated otherwise.
- Use the course discussion forum for questions and check whether the same question has already been answered before posting.